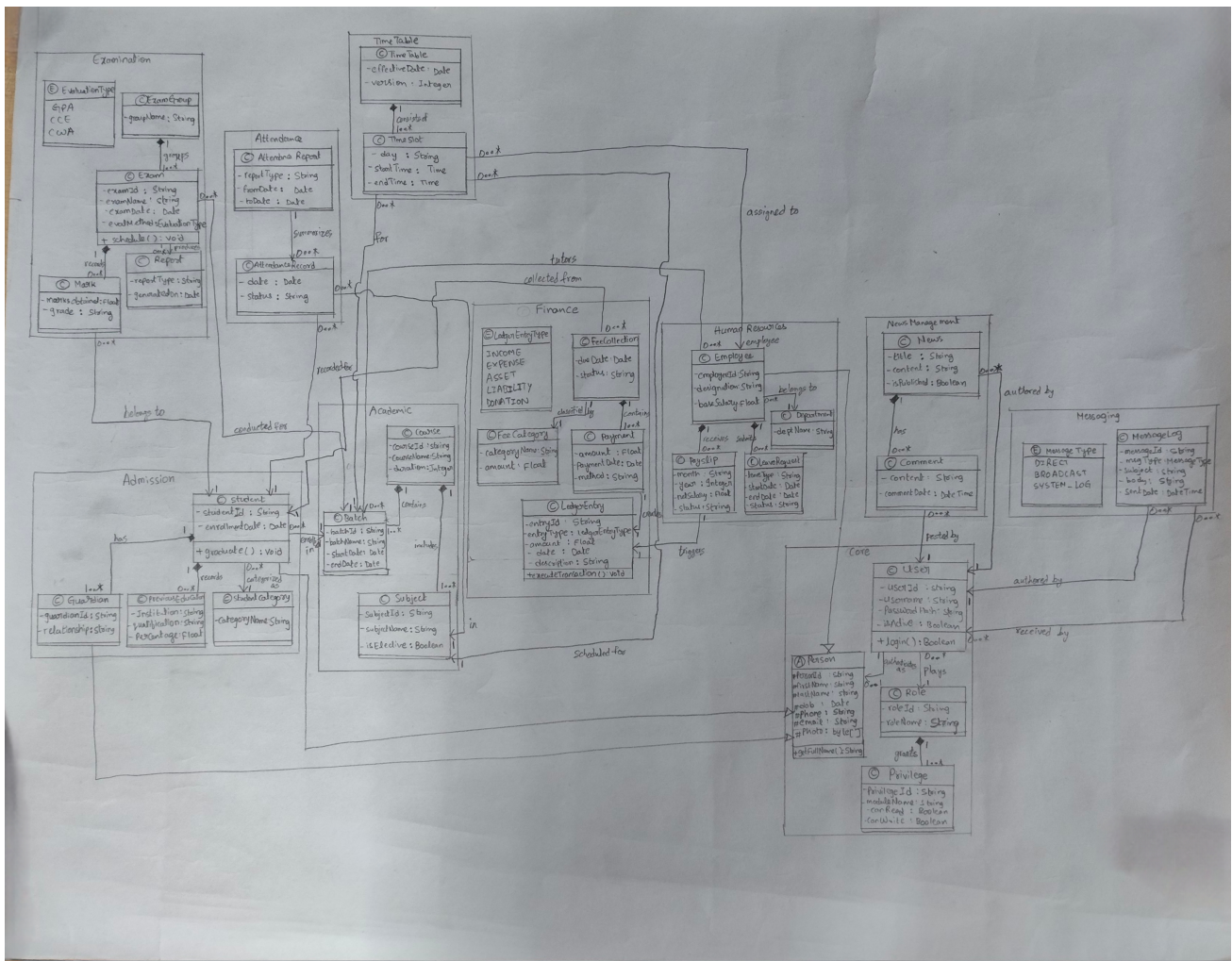


DASS Assignment-3: Task 1

Team Name: Unpaid Interns
Member 1: G.Phanindra (2024101068)
Member 2: Y.V.Kartikeya (2024111033)

Hand-drawn UML Class Diagram



Class Responsibilities: State and Behavior

The following table breaks down the 30 core domain classes of the IMS system, detailing the specific data each class is responsible for maintaining (State) and the methods or system interactions it supports (Behavior).

Class Name	Class State	Class Behavior
Person	• Maintains base personal data (ID, first/last name). • Holds contact info (email, phone) and photo.	• Implements <code>getFullName()</code>. • Provides foundational data inheritance for students and employees.
User	• Maintains system credentials (username, password hash). • Holds account active/inactive status.	• Implements <code>login()</code> for authentication. • Interacts with Role to determine system access limits.
Role	• Maintains role identifier and display name (e.g., “Admin”, “Teacher”).	• Groups multiple privileges together. • Assigns access boundaries to associated Users.
Privilege	• Maintains specific target module names. • Holds boolean flags for read, write, and delete permissions.	• Acts as the ruleset evaluated by the system before granting data access.
Course	• Maintains course metadata (ID, name, total duration).	• Aggregates Subjects and manages the lifecycle of multiple Batches.
Batch	• Maintains specific timeline data (start date, end date). • Holds the batch identifier/name.	• Groups Students together for scheduling and academic tracking.
Subject	• Maintains the subject name and ID. • Holds a boolean flag indicating if it is an elective.	• Links to TimeSlots for scheduling and AttendanceRecords for tracking.
Student	• Maintains specific enrollment dates. • Holds current active/graduated status.	• Implements <code>graduate()</code> to alter system state. • Interacts with Batches, Exams, and FeeCollections as the primary user entity.
Guardian	• Maintains relationship type to the student (e.g., “Parent”, “Uncle”).	• Acts as the designated emergency or financial contact for a Student.
PreviousEducation	• Maintains past academic records (institution name, qualification, percentage).	• Provides historical context required during the admission approval process.
StudentCategory	• Maintains category definitions (e.g., “International”, “Scholarship”).	• Used by the system to apply specific business logic (like fee discounts).

Class Name	Class State	Class Behavior
ExamGroup	• Maintains the collective group name for a set of exams.	• Aggregates individual Exams to generate consolidated performance reports.
Exam	• Maintains exam name, scheduled date, and evaluation type (Enum).	• Implements <code>schedule()</code> to lock in dates. • Evaluates Marks based on GPA, CCE, or CWA rules.
Mark	• Maintains the numerical score obtained by a student. • Holds the calculated letter grade.	• Links a specific Student's performance directly to an Exam.
Report	• Maintains the report type classification and generation timestamp.	• Summarizes raw Mark and Exam data into analytical, exportable formats.
Employee	• Maintains job designation and base salary figures.	• Submits LeaveRequests and tutors Batches. • Receives generated Payslips.
Department	• Maintains the departmental name (e.g., "Science", "Administration").	• Logically groups Employees for organizational and reporting purposes.
Payslip	• Maintains monthly payout figures (net salary). • Holds current approval/payment status.	• Triggers financial LedgerEntries upon approval to finalize payroll.
LeaveRequest	• Maintains requested start/end dates and leave type. • Holds the current workflow status (pending, approved, rejected).	• Interacts with HR approval workflows to adjust employee availability.
AttendanceRecord	• Maintains the specific date and the presence/absence status.	• Logs daily tracking for a Student against a specific Subject.
AttendanceReport	• Maintains a target date range (from/to) and report type.	• Aggregates Records to highlight truancy or calculate attendance percentages.
FeeCategory	• Maintains the fee name (e.g., "Tuition") and standard monetary amount.	• Acts as the template for generating mass fee collections.
FeeCollection	• Maintains the system due date and overall collection status.	• Tracks pending Payments and identifies defaulting Students.
Payment	• Maintains the paid amount, transaction date, and payment method.	• Reconciles outstanding FeeCollections and triggers corresponding LedgerEntries.

Class Name	Class State	Class Behavior
LedgerEntry	• Maintains transaction amount, date, description, and EntryType Enum.	• Implements <code>executeTransaction()</code>. • Centralizes all income, expenses, assets, and liabilities.
MessageLog	• Maintains message subject, body text, timestamp, and MessageType.	• Routes direct communications and system-wide broadcasts between Users.
TimeTable	• Maintains the schedule's effective date and version number.	• Aggregates TimeSlots and runs conflict-detection algorithms.
TimeSlot	• Maintains the day of the week, start time, and end time.	• Binds an Employee, a Subject, and a Batch to a specific calendar block.
News	• Maintains the article title, content body, and published boolean status.	• Broadcasts institutional announcements to the Dashboard.
Comment	• Maintains the text content of the feedback and a timestamp.	• Captures and displays User reactions linked directly to a News article.

Dynamic Diagrams

Sequence Diagrams

The following sequence diagrams model three distinct operational flows within the IMS, demonstrating the interaction between actors, domain entities, and system services.

Flow 1: Student Admission Process

This flow models a new student being admitted, assigned to an academic batch, and having their system credentials and welcome alerts generated.

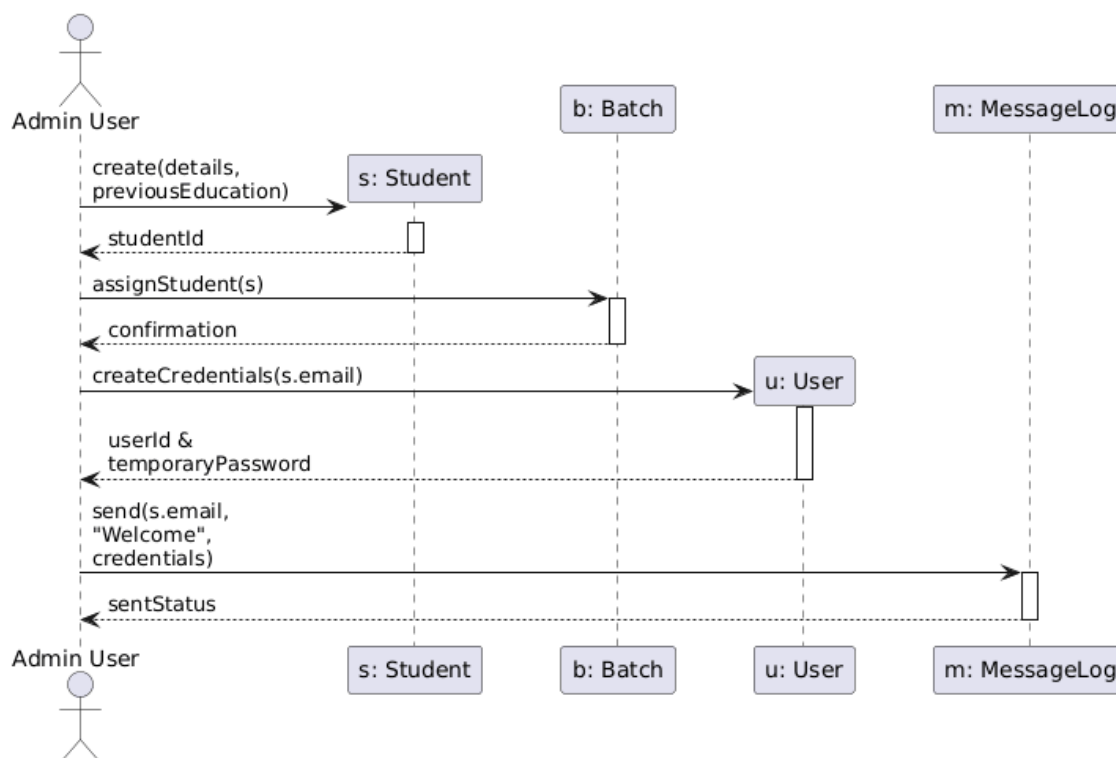


Figure 1: Flow 1

Flow 2: Student Attendance Marking

This workflow models the Attendance module, tracing a teacher fetching a batch roster, iterating through the enrolled students, and creating individual attendance records linked to a specific subject.

Flow 3: Fee Payment Processing

This sequence maps the Finance module's payment process, detailing how a student fee payment is recorded and automatically triggers a centralized ledger entry.

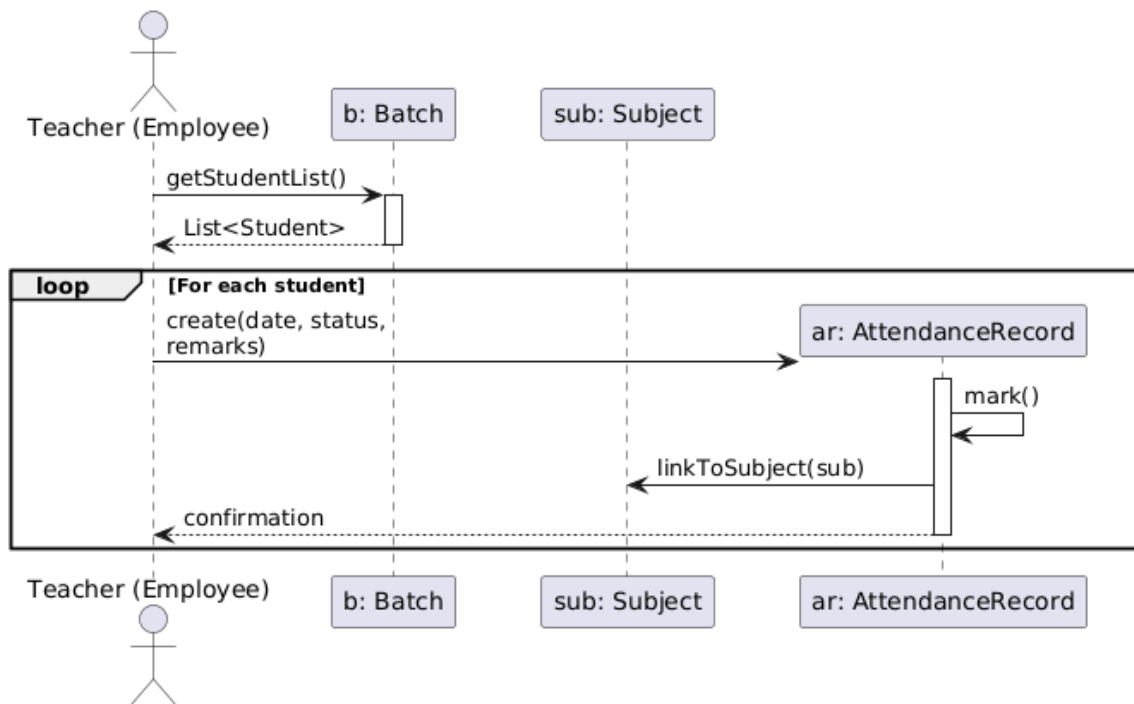


Figure 2: Flow 2

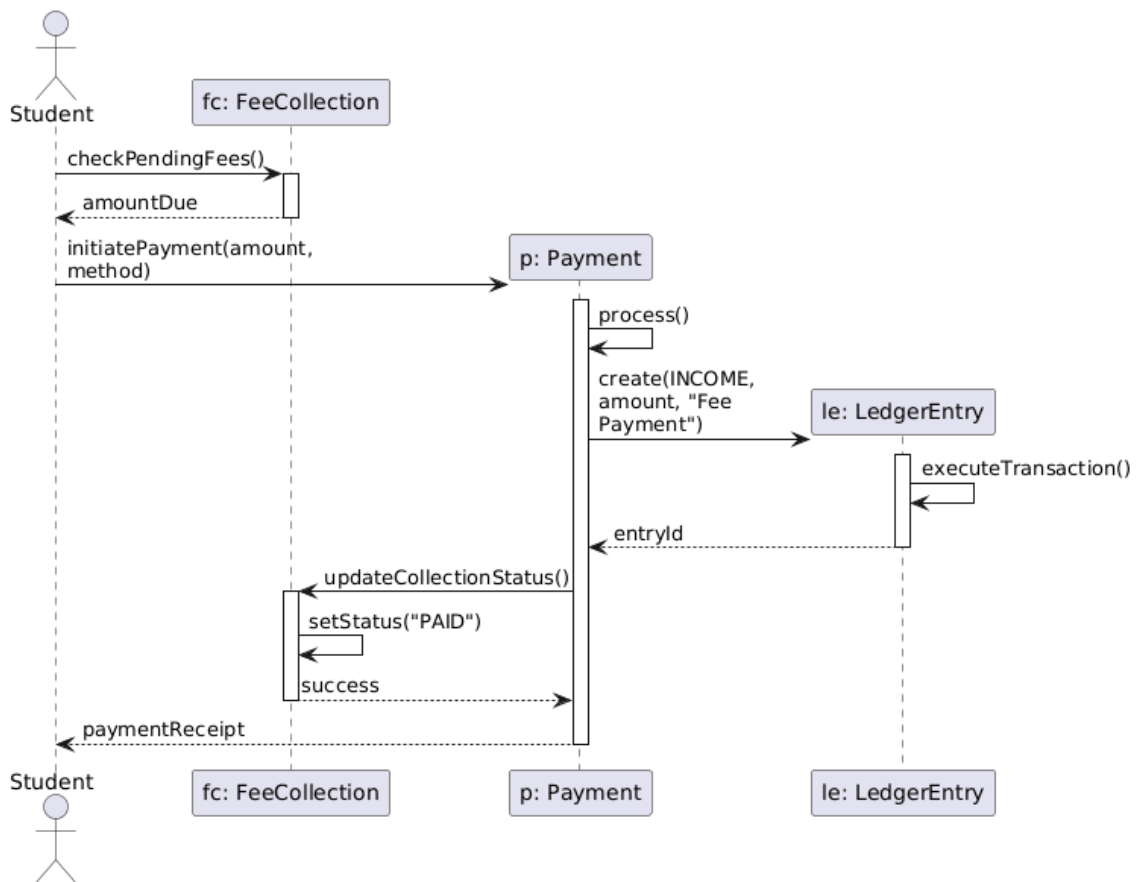
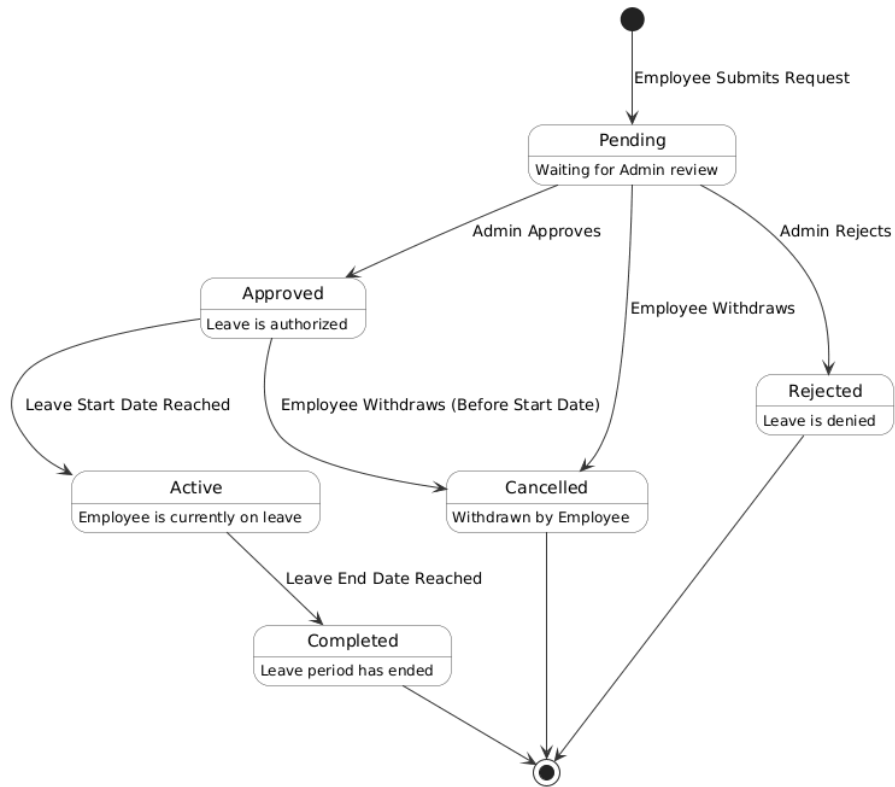


Figure 3: Flow 3

State Diagram

Entity: Leave Request (Human Resources Module)

The following state machine diagram models the strict lifecycle of a Leave Request entity, ensuring valid transitions from submission to completion or cancellation.



Design Narrative & Analysis

The Institute Management System (IMS) domain model was architected to prioritize modularity, maintainability, and clear data boundaries. The design balances key object-oriented design principles as follows:

Separation of Concerns (SoC)

The design strictly separates the core business domain from the presentation and application layers. By purposefully excluding UI controllers (like the Dashboard) and data-entry artifacts (like customizable admission or payroll forms) from the conceptual class diagram, the domain entities remain strictly focused on state and business rules. This ensures that changes to the user interface do not force recompilation or refactoring of core data structures.

High Cohesion

Classes are heavily encapsulated around single, well-defined purposes. For example, rather than scattering financial logic across **Student**, **Employee**, and **FeeCollection**, all financial tracking is highly cohesive within the **LedgerEntry** class. Similarly, the **Exam** class takes full ownership of evaluation methodologies (via the **EvaluationType** enumeration) and mark recording, ensuring that examination logic is not leaked into the broader **Academic** module.

Low Coupling

The system minimizes direct, rigid dependencies between disparate modules. For instance, the authorization system utilizes a **Role** class as an intermediary between a **User** and their specific **Privileges**. Furthermore, **Payment** objects interact with the finance system by generating a standardized **LedgerEntry** rather than directly manipulating total organizational asset values. This modularity allows the HR or Finance modules to be updated or swapped with minimal cascading effects on the rest of the system.

Law of Demeter (Principle of Least Knowledge)

The model adheres to the Law of Demeter by ensuring objects only interact with their immediate, closely related neighbors. Deep navigational chains (e.g., `Dashboard.getCourse().getBatch().getStudent().getMark()`) are avoided by delegating behaviors. Instead of external classes fishing for deep data, aggregating classes like **Report** and **AttendanceReport** exist specifically to query and summarize data directly from their immediate compositional children (**Exam** and **AttendanceRecord**, respectively), limiting structural exposure.

Pattern & Anti-Pattern Analysis

Design Patterns Utilized

1. Polymorphism (GRASP Pattern)

The system is required to support varied evaluation methods, specifically GPA, CCE, and CWA. Rather than hardcoding these grading algorithms directly into the **Exam** class using rigid conditional logic (e.g., extensive if-else blocks), we utilized the GRASP Polymorphism pattern. By abstracting the evaluation strategy into distinct types, the calculation behavior is handled polymorphically at runtime without altering the core entities.

2. Information Expert (GRASP Pattern)

The Information Expert principle dictates that a responsibility should be assigned to the class that has the most information necessary to fulfill it. We applied this strictly to the **LedgerEntry** class within the Finance module. Instead of external classes extracting raw data to calculate balances, **LedgerEntry** inherently holds the transaction amounts, types, and execution methods, making it the sole expert responsible for processing and centralizing the financial state.

Design Anti-Patterns Avoided

1. The God Object (The Blob)

A common pitfall in system design is creating a monolithic controller class (e.g., a massive **SystemManager** or **Dashboard** class) that knows too much or does too much. We strictly avoided this by decentralizing control. The **Dashboard** was relegated to the presentation layer, and domain logic was tightly scoped. For example, financial operations are entirely encapsulated within the **LedgerEntry** and **FeeCollection** classes, rather than being managed by a centralized academic or user manager.

2. Poltergeists (Useless Classes)

Poltergeists are transient classes with limited responsibilities and short lifecycles whose only real purpose is to invoke methods in other classes—cluttering the architecture. Early iterations of the design included transient UI boundaries like **AdmissionForm** and **PayrollForm**. We actively avoided this anti-pattern by removing these stateless pass-through classes from the core domain model, mapping their data directly to the robust entity classes (**Student** and **Employee**) that actually maintain system state.

Reflection

Strongest Aspects

1. Centralized and Scalable Financial Ledger

Instead of scattering financial data across isolated entities for incomes, expenses, assets, liabilities, and donations, the design consolidates all monetary tracking into a single **LedgerEntry** class using an enumeration strategy. This streamlines the generation of balance sheets, prevents data duplication, and makes the Finance module robust and highly cohesive.

2. Polymorphic Evaluation Strategy

By abstracting the grading algorithms into an **EvaluationType** linked to the **Exam** class, the system inherently supports GPA, CCE, and CWA frameworks without hardcoding them into the core entities. This prevents the examination logic from becoming bloated with complex conditional statements and ensures maximum extensibility.

Weakest Aspects

1. Simplistic Scheduling Constraints

While the **TimeTable** and **TimeSlot** classes successfully model time allocations and basic employee workload limits, the design lacks a dedicated rule-engine class. Handling complex, real-world scheduling constraints—such as specific laboratory equipment requirements, overlapping elective pools, or dynamic teacher unavailability—would require significant expansion of this module.

2. Coarse-Grained Authorization Boundaries

The **Role** and **Privilege** classes efficiently handle module-level access control (e.g., granting read/write access to the Examination module). However, the model lacks a natural structure for fine-grained, row-level security. For example, inherently restricting a teacher so they can only modify **Marks** for students in their assigned **Batch** would require additional contextual validation logic outside the core structural model.